Parallel Morphological Image Processing

Accelerating morphological operations via C++17 and OpenMP

Davide Lombardi

July 20, 2026

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.

Introduction

o

- **Mathematical Morphology:** framework for geometric feature extraction and image restoration
- **Targeted Operations:**
  - *Fundamental:* Erosion, Dilation
  - *Compound:* Opening, Closing
- **Goal:** resolve computational bottlenecks in high-resolution image processing
- **Technology Stack:** `C++17` and `OpenMP` for thread orchestration
- **Dataset & Benchmarks:** BSDS500
  - Preliminary grayscale conversion
  - 2.0x and 4.0x upscaling to evaluate scalability

Context & Architectural Limits

o

Operator Logic

- **Erosion:** minimum extraction in a defined spatial neighborhood
- **Dilation:** maximum extraction in a defined spatial neighborhood
- **Opening:** Erosion followed by Dilation
- **Closing:** Dilation followed by Erosion

The Memory Wall & Complexity

- Naive execution with 2D stencils strictly bound by $\mathcal{O}(M \times N \times K^2)$ complexity
- **Compound operations** implemented sequentially (double-pass approach)
- **System Overhead:** destruction of *temporal locality* and cache saturation between passes

Hardware Optimization: Pixel-Level Parallelism

o

- Spatial stencil processing is inherently independent for each pixel
- **Loop Collapsing (`collapse(2)`):** logical fusion of X and Y image axes
  - Expands the iteration space and distributes workload with fine granularity
  - Prevents *thread starvation* at boundaries
- **Memory Safety (`default(none)`):** forced explicit scoping
  - Thread-local allocation for temporary variables (e.g., `min_val`)
  - Avoids synchronization overhead (zero locks or atomics) in the critical loop
- **Static Scheduling (`schedule(static)`):** deterministic work distribution
  - Zero runtime overhead, ideal for uniform per-pixel workloads (invariant $K \times K$ footprint)
  - *Note:* dynamic chunking later explored

Algorithmic Optimization: 1D Separable Kernels

o

- **The Dominant Lever:** decomposition of the $K \times K$ stencil into two independent passes (Horizontal $\rightarrow$ Vertical)
- **Computational Reduction:** transition from $\mathcal{O}(K^2)$ to linear $\mathcal{O}(2K)$
  - Using a 15×15 kernel, base operations per pixel drop from 225 to just 30

The Architectural Trade-off (Cache Asymmetry)

- **Horizontal Pass:** highly efficient, exploits prefetching and RAM *spatial locality* (row-major layout)
- **Vertical Pass:** introduces severe cache misses, forcing the memory bus to work column-wise

Producer-Consumer Pipeline

o

- **The Fork-Join Problem:** pure 1D Closing requires 4 global passes, doubling heap footprint and introducing strict synchronization barriers
- **OpenMP Pipeline Design:**
  - Single partitioned parallel region where half the threads act as Producers and half as Consumers
  - Fine-grained chunk synchronization via `PaddedFlag` array to eliminate false sharing
- **Architectural Benefits:** execution overlap and keeps pixels in cache (*temporal locality*)

Performance Results

o

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

Strong & Weak Scalability Analysis

o

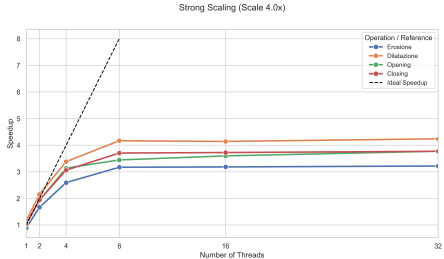

Figure: Strong Scaling (Fixed Workload)

- Asymptotic saturation at **3.5x–4.0x** with 4 physical cores
- Perfectly reflects *Amdahl's Law* (unavoidable pure sequential fraction)

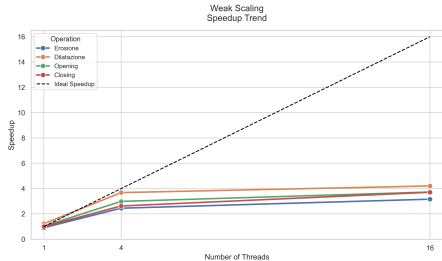

Figure: Weak Scaling (Scaled Workload)

- Constant execution times when doubling workload and cores
- Confirms excellent hardware resource distribution

Algorithmic Reduction: 1D vs 2D Performance

o

- **Direct Impact:** transitioning to linear $\mathcal{O}(2K)$ complexity acts as the true performance multiplier
- **Kernel Scaling (K):**
  - At low K values, the gap is minimal
  - At high K values, the 2D approach collapses under quadratic load

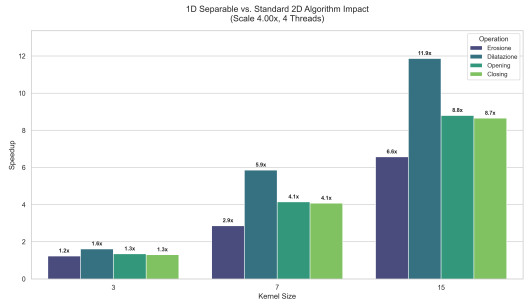

Figure: Speedup of 1D Separable Kernels vs 2D baseline

Producer-Consumer Pipeline Efficiency

o

- **Focus:** specific optimization for Opening and Closing
- **Peak Efficiency:** yields up to a **1.67x** performance boost over the standard parallel baseline
- **High-Concurrency Contention:**
  - Active *spin-wait* loops saturate physical cores under high logical thread counts
  - Cross-chunk dependencies amplify synchronization overhead and cause thread starvation

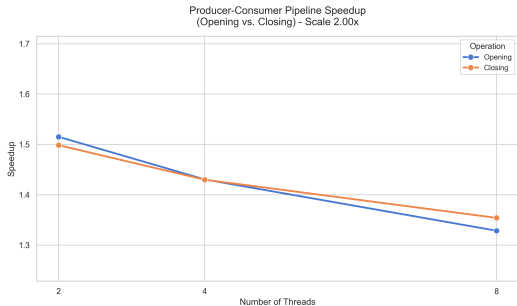

Figure: Pipeline speedup for compound operations

Other Strategies Tested

o

- **SIMD Vectorization (`#pragma omp simd`):**
  - No positive impact on core loops
  - Failure due to compiler auto-vectorization (already optimal) and min/max conditional branches

- **Dynamic Loop Scheduling (`schedule(dynamic)`):**
  - Evaluated on the baseline parallel version to improve load balancing
  - Performance remained virtually identical to the default `static` approach

Peak Performance & Future Developments

o

Optimal Configuration

22x Peak Speedup

Mixed 1D Separable + OpenMP architecture ($K = 15$, 4 Threads) over 2D Baseline.

- **Hardware/Software Conclusion:** bare multithreading can't save a flawed algorithm. Hardware/software integration is the only path to raise high performance
- **Heterogeneous Computing (Future Work):**
  - Porting the logic to GPU architectures (CUDA)

Visual Results

o

Original

Erosion

Dilation

Opening

Closing

Parallel Morphological Image Processing

Thank you for listening!

Any questions?